



Bölüm 5: Dizgi Algoritmaları

Algoritmalar



Dizgi Algoritmaları

- Metinlerle dolu bir dünyada yaşıyoruz.
 - E-postalar, mesajlar, sosyal medya paylaşımları, haber metinleri...
- Bilgisayarlarımızda her gün sayısız metinle karşılaşırız.
- Peki, bu metinler nasıl düzenlenir ve analiz edilir?
- Dizgi (String) algoritmaları,
 - metinlerde arama,
 - değiştirme,
 - karşılaştırma gibi işlemleri gerçekleştirir.



Dizgi Eşleştirme Algoritmaları

- Brute Force (Kaba Kuvvet):
 - Metinde *her konumdaki* değer, örüntü ile eşleştirmek için kontrol edilir.
 - Maksimum sayıda karşılaştırma gerektirebilir.
- Knuth-Morris-Pratt (KMP)
 - Başlangıçta *tablo* oluşturularak arama süresi azaltılır,
 - Karakter karşılaştırmalarını azaltarak hızlı çalışır.
- Boyer-Moore
 - Uzun aramalarda etkili. Kötü karakter ve iyi sonek kaydırma kurallarını kullanır.
- Rabin-Karp Algoritması
 - Olasılıksal bir algoritma. *Hashing* kullanır.



Dizgi Sıkıştırma Algoritmaları

- Sıralı Sıkıştırma Kodlaması (Run Length Encoding)
 - Aynı veri değerleri tek bir *değer ve sayı* olarak saklanır.
 - Tekrar eden değerler yerine tekrar eden veri sayısı saklanır.
- Lempel-Ziv-Welch (LZW)
 - GIF gibi formatlarda kullanılan sözlük tabanlı sıkıştırma algoritması.
 - Tekrar eden örüntüleri *sözlük* oluşturarak kısa sembollerle temsil eder.
 - Dinamik bir sözlük kullanarak sıkıştırma sağlar.

String Matching





String Naive Algoritması

- Metin içinde belirli bir örüntüyü (*pattern*) arayan basit bir algoritma.
- *Naive* (Saf) olarak adlandırılır çünkü basit bir yaklaşım kullanır.
- Ortalama ve en kötü durumda $O(m\ n)$ zaman karmaşıklığına sahiptir.
 - (m: örüntü uzunluğu, n: metin uzunluğu)

İşleyiş



- Metindeki her karakter için örüntünün ilk karakterinin eşleşip eşleşmediği kontrol edilir.
- Eşleşen karakterlerin tümü için örüntünün tam olarak eşleşip eşleşmediğini kontrol edilir.
 - Eşleşme Durumu: eşleşme pozisyonu rapor edilir.
 - Eşleşmeme Durumu: sonraki karakterler kontrol edilir.
- Metindeki tüm karakterler için adımlar tekrarlanır.



Brute Force - Naive

T	H	I	S		I	S		A		S	I	M	P	L	E		E	X	A	M	P	L	E
S	I	M	P	L	E																		
	S	I	M	P	L	E																	
		S	I	M	P	L	E																
			S	I	M	P	L	E															
				S	I	M	P	L	E														
					S	I	M	P	L	E													
						S	I	M	P	L	E												
							S	I	M	P	L	E											
								S	I	M	P	L	E										
									S	I	M	P	L	E									
										S	I	M	P	L	E								

Brute Force - Naive



HELLO WORLD



Brute Force - Naive





Brute Force - Naive





Brute Force - Naive



Brute Force - Naive





Brute Force - Naive





Brute Force - Naive





Brute Force - Naive





Brute Force - Naive





Brute Force - Naive







Knuth-Morris-Pratt (KMP) Algoritması

- Metin içinde belirli bir örüntüyü bulmak için kullanılır.
- *Donald Knuth, Vaughan Pratt* ve *James H. Morris* tarafından geliştirilmiştir.
- Ortalama ve en kötü durumda $O(m + n)$ zaman karmaşıklığına sahiptir.
 - (m: örüntü uzunluğu, n: metin uzunluğu)



İşleyiş

- Ön İşleme: Örüntü içindeki her karakter için,
 - eşleşme durumunda geri dönecek pozisyonları belirleyen,
 - en uzun önek(*suffix*) eşleşmesini bulan bir tablo oluşturulur.
- Örüntü Arama: Metin içinde arama yapılırken,
 - örüntü ile eşleşmeyen karakterlerde geri dönecek pozisyonlar,
 - tablodan elde edilen geri dönüş değerleri kullanılarak hesaplanır.
- Eşleşme Kontrolü: Eşleşen karakterlerin tümü için örüntünün tam olarak eşleşip eşleşmediği kontrol edilir.

Örnek

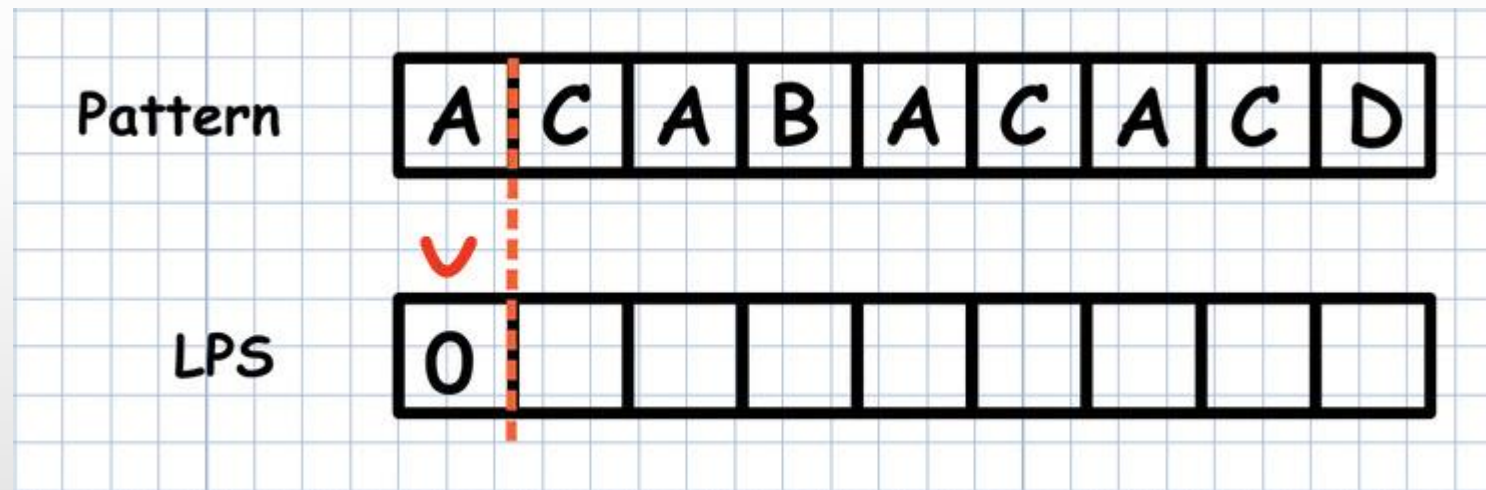


- Metin: "ababcababababd"
- Örüntü: "ababd"
- Örüntü Tablosu:
 - a: 0, b: 0, a: 1, b: 2, d: 0

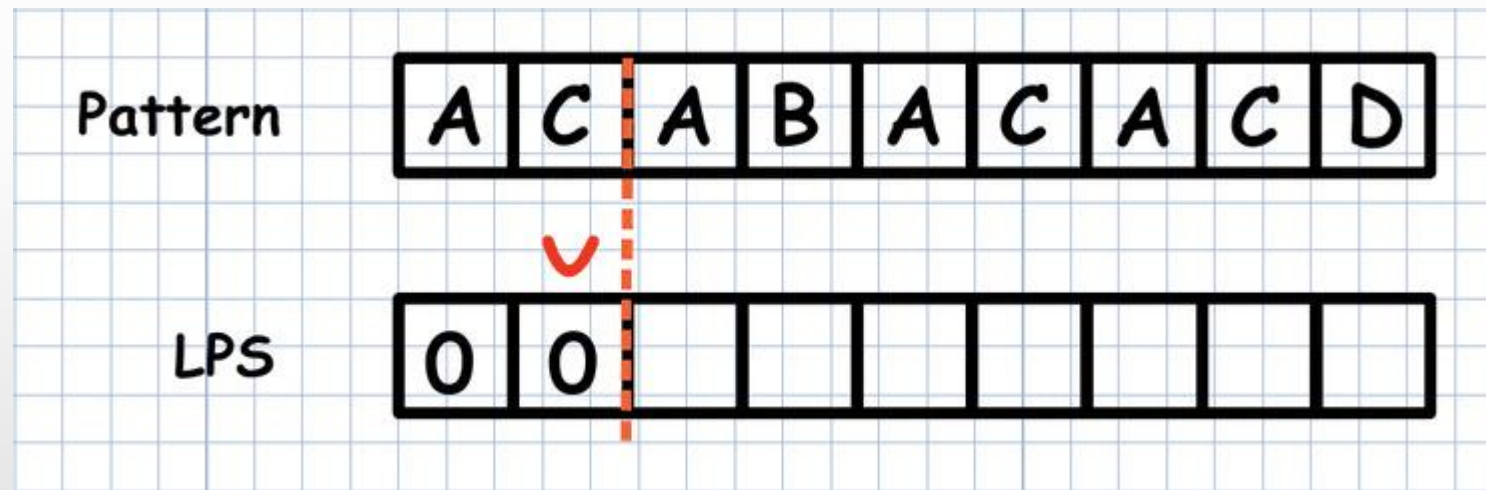


Knuth Morris Pratt

- *Longest Proper Prefix*

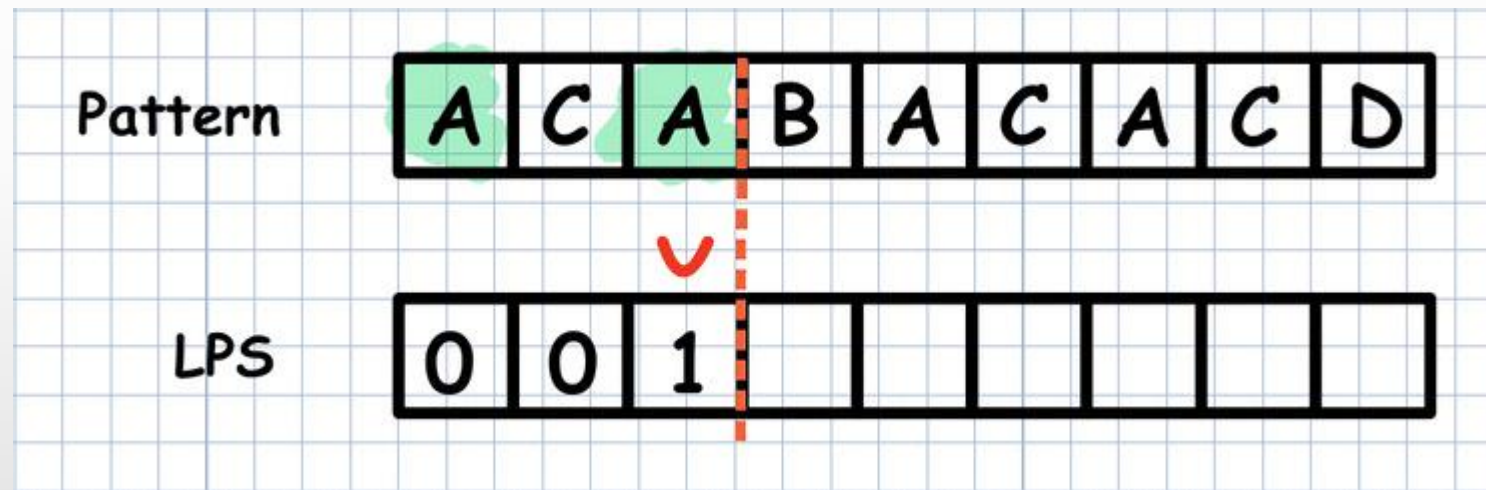


Knuth Morris Pratt



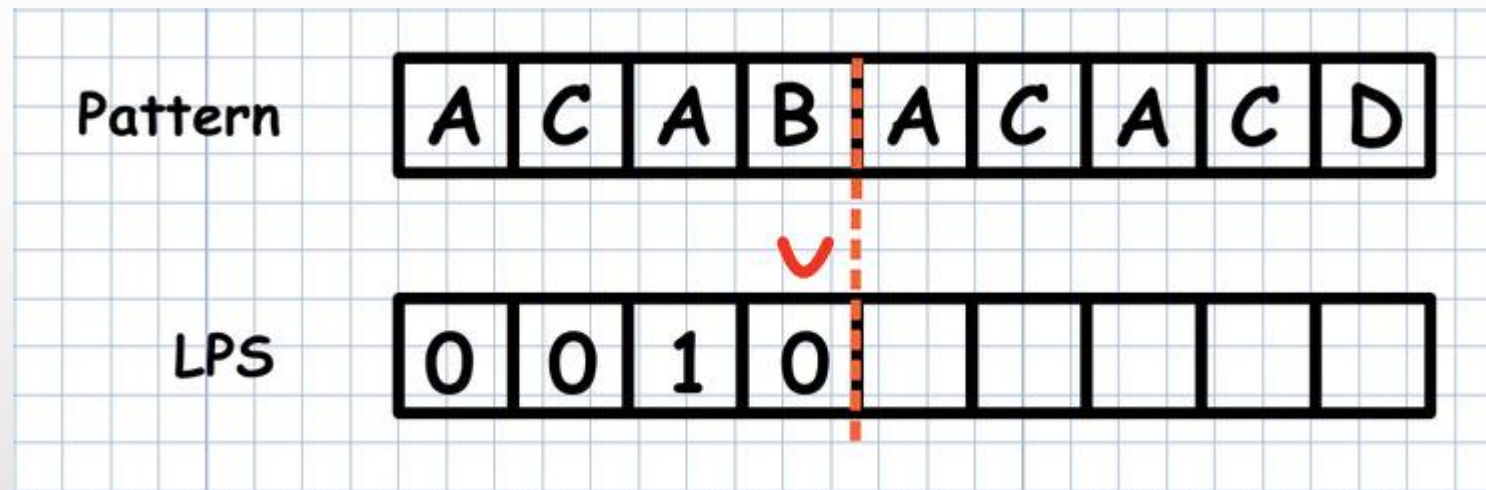


Knuth Morris Pratt



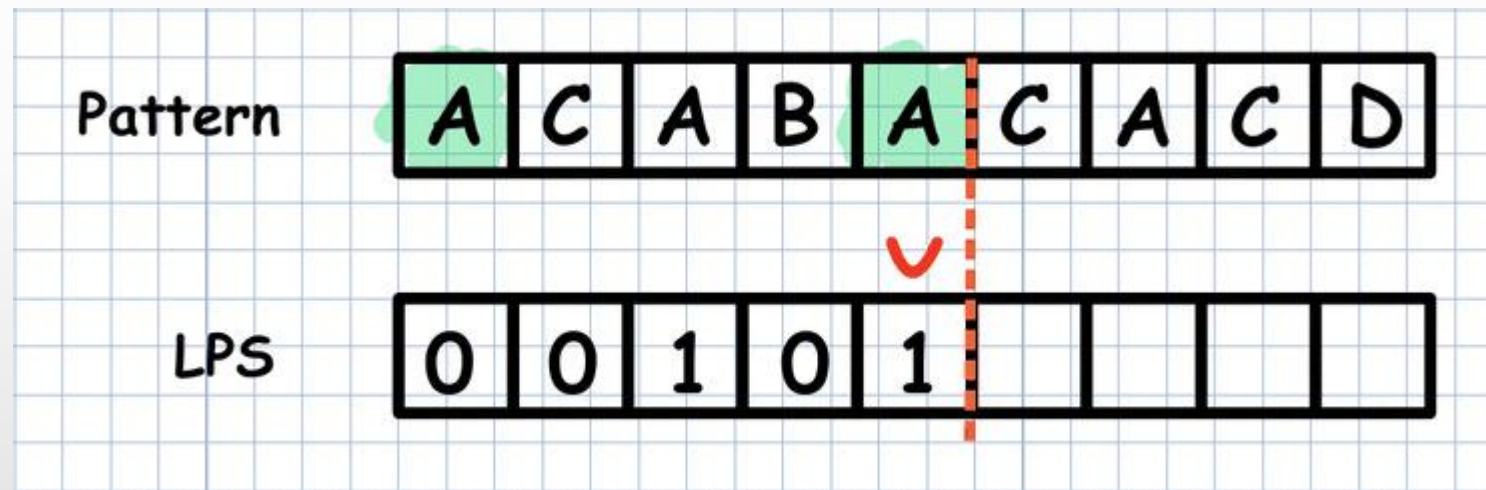


Knuth Morris Pratt



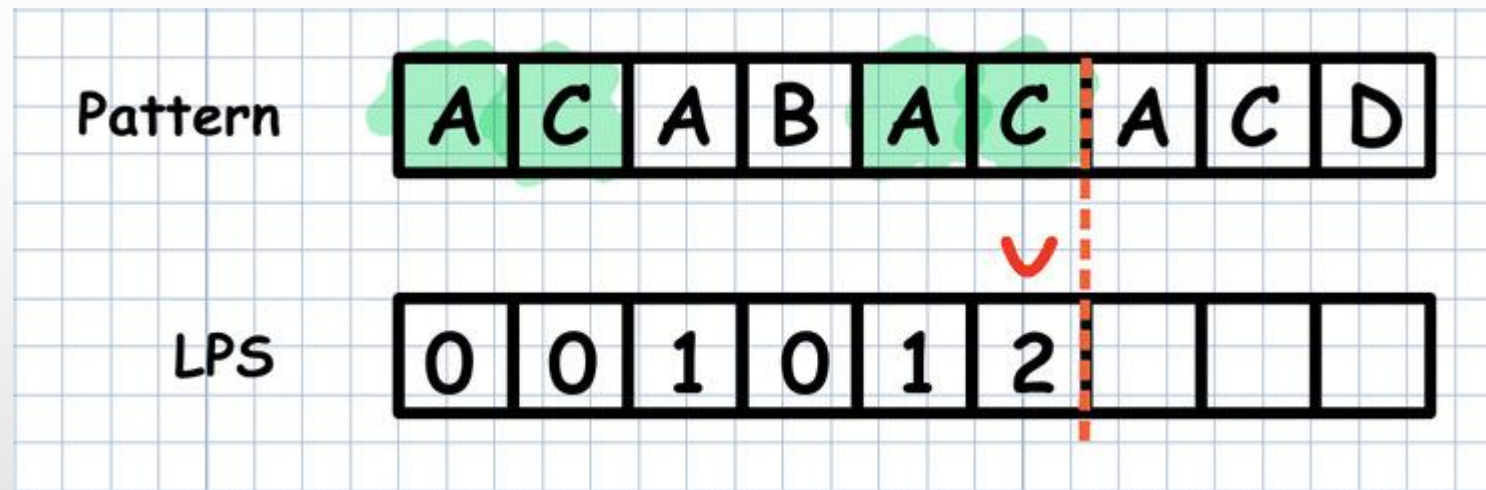


Knuth Morris Pratt





Knuth Morris Pratt





Knuth Morris Pratt

Pattern	A	C	A	B	A	C	A	C	D
LPS	0	0	1	0	1	2	3		



Knuth Morris Pratt

Pattern	A	C	A	B	A	C	A	C	D
LPS	0	0	1	0	1	2	3	2	

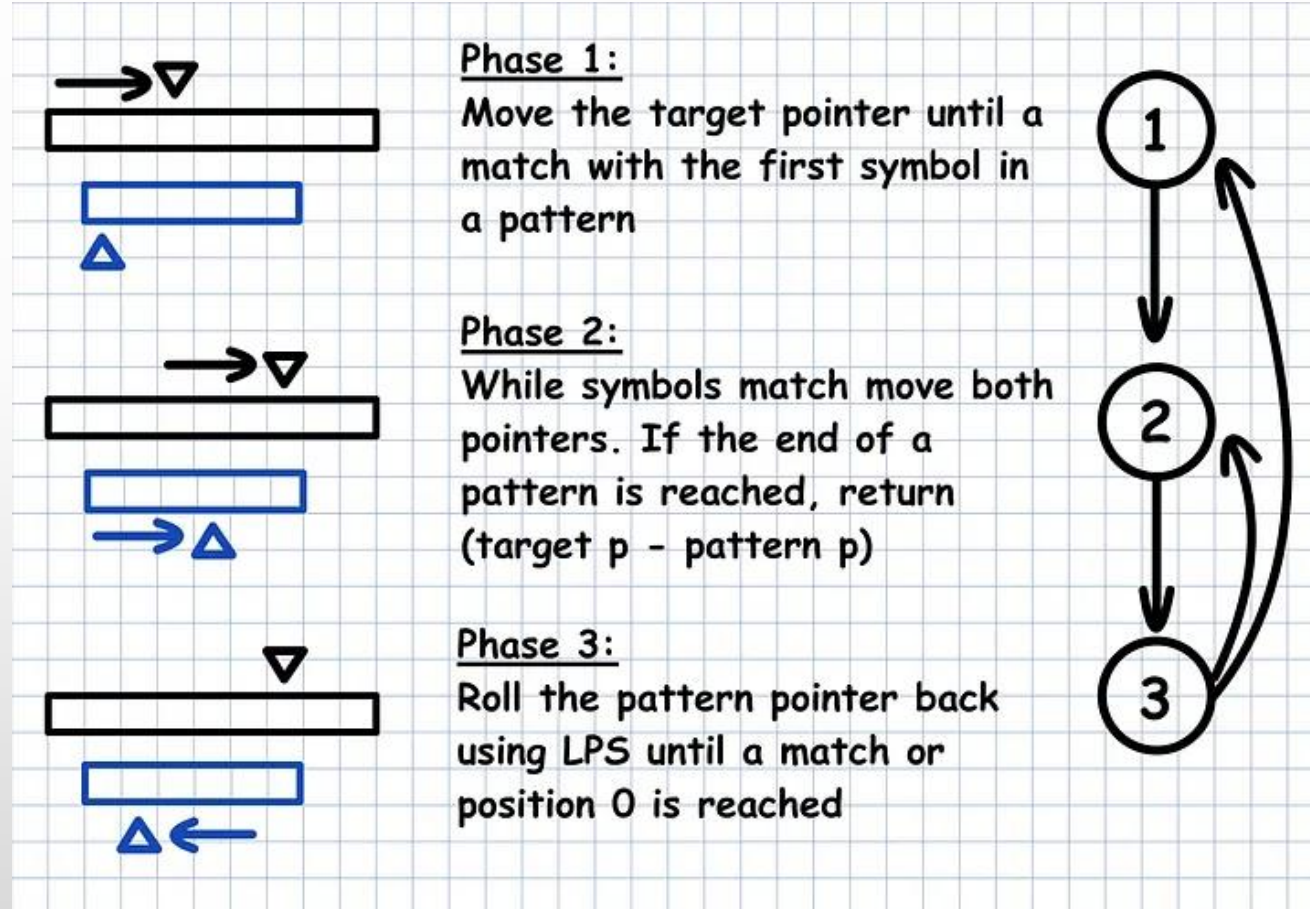


Knuth Morris Pratt

Pattern	A	C	A	B	A	C	A	C	D
LPS	0	0	1	0	1	2	3	2	0



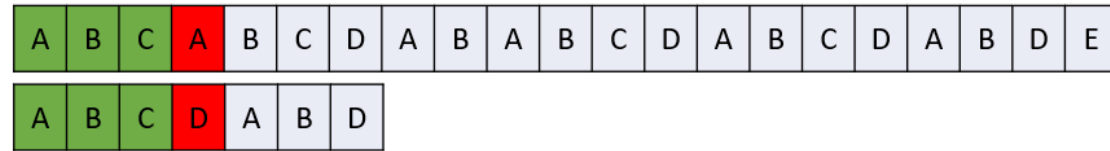
Knuth Morris Pratt



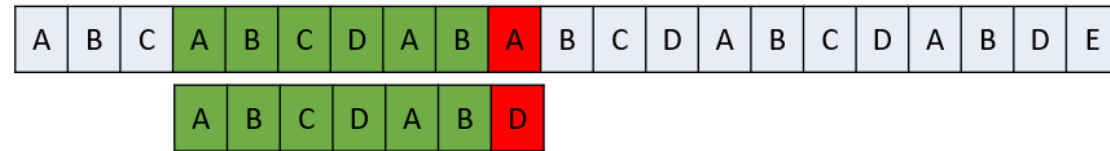


Knuth Morris Pratt

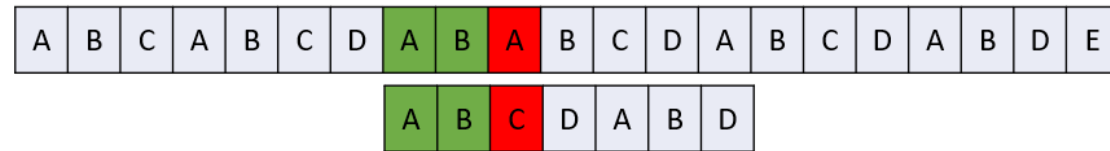
STEP 1



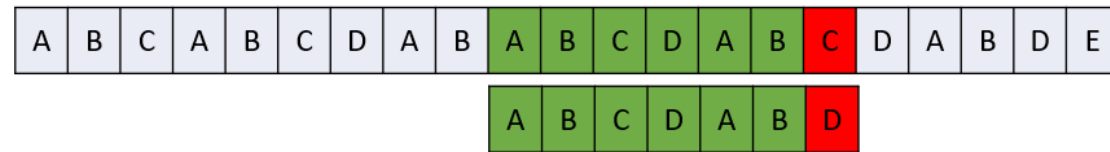
STEP 2



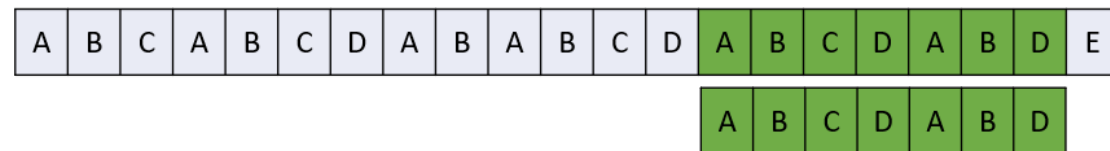
STEP 3



STEP 4



STEP 5







Boyer-Moore Algoritması

- Metin içinde belirli bir örüntüyü arar.
- *Robert S. Boyer* ve *J Strother Moore* tarafından geliştirilmiştir.
- En İyi (Best Case): Aranan desenin metinle hiç uyuşmadığı ve maksimum atlamaların yapıldığı durum. $O(n/m)$
- Ortalama (Average Case): Tipik metin aramalarında karakterlerin çoğunun okunmadan geçilmesiyle elde edilir. $O(n)$
- *En Kötü (Worst Case): Desen ve metnin çok benzer olduğu, neredeyse her adımda karşılaştırma yapıldığı durum.*
- (n : metin uzunluğu, m : desen uzunluğu)

İşleyiş



- Ön İşleme: Örüntü içindeki her karakter için eşleşme durumunda geri dönülecek pozisyonları belirleyen bir tablo oluşturulur.
- Arama: Metin içinde örüntüyü ararken, eşleşmeyen karakterlerde tablodan yararlanarak geri dönülecek pozisyonlar hesaplanır.
- Eşleşme Kontrolü: Eşleşen karakterlerin tümü için örüntünün tam olarak eşleşip eşleşmediği kontrol edilir.
- Kötü Karakter Kaydırma Kuralı: Eşleşmeyen karakterin örüntü içindeki en sağdaki konumu dikkate alınarak kaydırma yapılır.
- İyi Sonek Kuralı: Eşleşmiş sonekin örüntü içinde başka bir yerde bulunup bulunmadığına göre kaydırma yapılır.



Örnek

- Metin: "abccbaabccbaabcbcabbbababcabc"
- Desen: "abcbcabbbababcabc"
- Desen Tablosu:
 - a: 10, b: 8, c: 7
- Sonuç:
 - Pozisyon 12: "abcbcabbbababcabc"

Boyer Moore



H	E	L	L	O												
L	O		L	O	E	L	L	O		O		H	E	L	L	O

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO
LO LLOELLOHELLO

Boyer Moore



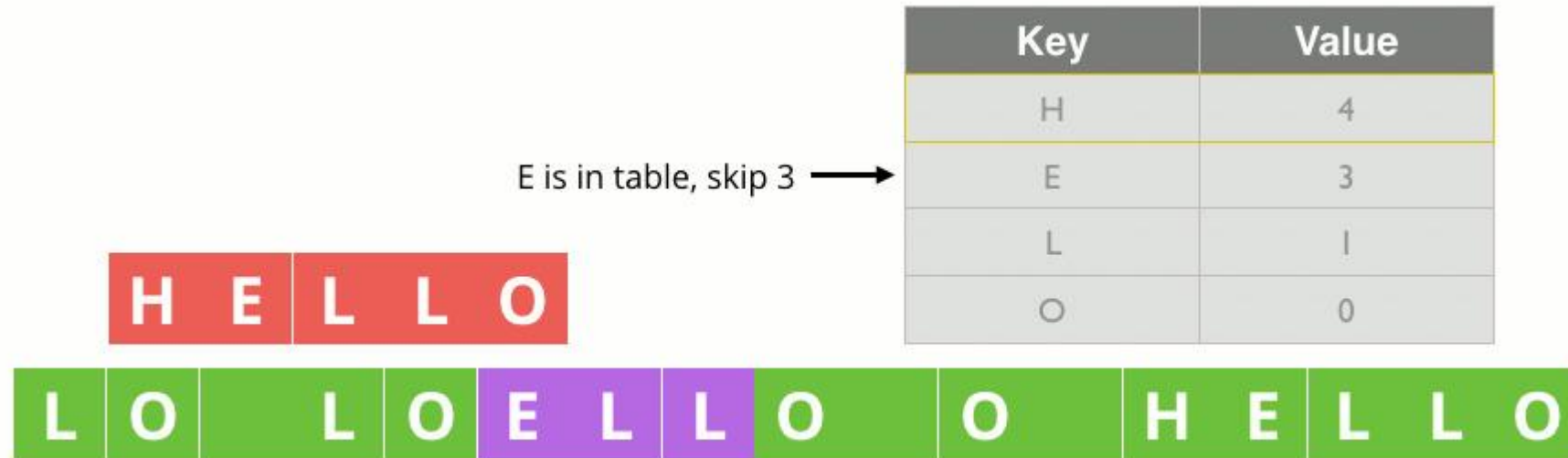
Couldn't match, move on

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



Boyer Moore



HELLO
LO LOELLO O HELLO

Boyer Moore



HELLO
LO LOELLOLO HELLO

Boyer Moore



HELLO
LO LOELOLO OHELLO

Boyer Moore



HELLO
LO LOELOLO OHELLO

Boyer Moore



HELLO
LO LOELLO OHELLO

Boyer Moore



Couldn't match, move on

Boyer Moore



HELLO
LO LOELLO O HELLO

Boyer Moore



HELLO
LO LO E L L O O H E L L O

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO
LO LOELLO O HELLO

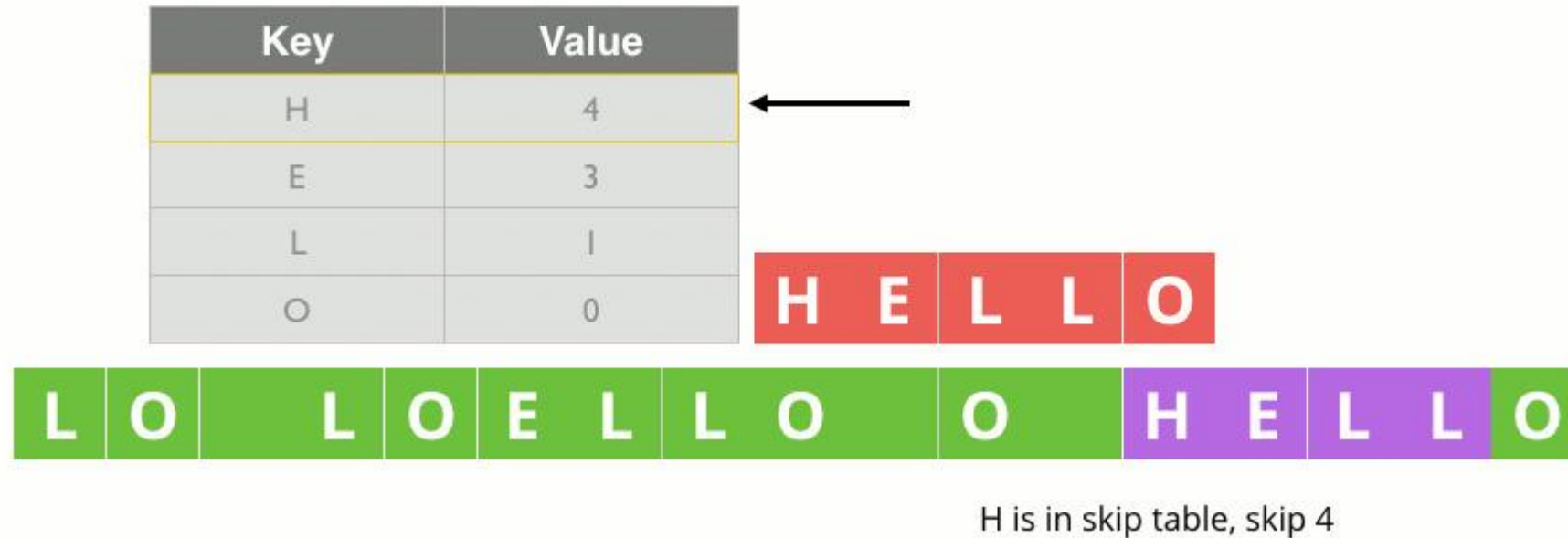
Boyer Moore



HELLO
LO L O E L L O O H E L L O



Boyer Moore



Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO
LO L O E L L O O H E L L O

Boyer Moore



HELLO

LOELLO

Boyer Moore



✓ H E L L O
L O L O E L L O O H E L L O





Boyer Moore

Pattern

a	b	a	c	a	b
0	1	2	3	4	5

preprocessing the pattern

Last occurrence table

a	b	c	*
4	5	3	-1

* denotes any character not present in the table

Text

a	b	a	c	a	a	b	a	d	c	a	b	a	c	a	b	a	a	b	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

comparison counter → 1

1

a	b	a	c	a	b
---	---	---	---	---	---

2

a	b	a	c	a	b
---	---	---	---	---	---

3

a	b	a	c	a	b
---	---	---	---	---	---

4

a	b	a	c	a	b
---	---	---	---	---	---

5

a	b	a	c	a	b
---	---	---	---	---	---





Rabin-Karp Algoritması

- Metin içinde belirli bir örüntüyü bulmak için kullanılır.
- *Michael O. Rabin* ve *Richard M. Karp* tarafından geliştirilmiştir.
- En İyi (Best Case): Hash değerlerinin nadiren çakıştığı ve hızlıca tarama yapıldığı durum. $O(n + m)$
- Ortalama (Average Case): Pratik uygulamalarda görülen genel performans. $O(n + m)$
- En Kötü (Worst Case): Tüm hash değerlerinin çakıştığı (spurious hits) ve sürekli karakter kontrolü gerektiği durum. $O(nm)$
- (n: metin uzunluğu, m: örüntü uzunluğu)



İşleyiş

- *Hash*: Örüntü ve metin içindeki alt dizgelerin hash değerleri hesaplanır.
 - Eşleşme Kontrolü: Hash değerleri eşleşen alt dizgeler karşılaştırılır.
 - Doğrulama: Eşleşme olduğunda, karakter bazında doğrulanır.
 - Kaydırma ve Yeniden Hesaplama: Eşleşme olmadığında, yeni bir alt dizge seçilir ve hash değeri yeniden hesaplanır.
 - Tekrarlama: Tüm metin boyunca adımlar tekrarlanır.
-
- RLE ve LZW kayıpsız sıkıştırma algoritmalarıdır. Sıkıştırılmış veri açıldığında orijinal veri aynen elde edilir.



Örnek

- Metin: "abracadabra"
- Örüntü: "cad"
- İşleyiş:
 - Hash Değerleri: Metin: "abr", Örüntü: "cad"
 - Eşleşme Kontrolü: Hash değerleri eşleşmez.
 - Kaydırma ve Yeniden Hesaplama: Yeni alt dizge seçilir: "bra"



Rabin Karp

V U A T S

$$\underline{5} + 1 = 6$$

T S

$$2 + 3 = 5$$

1

Values

$$U = 1$$

$$T = 2$$

$$S = 3$$

$$A = 4$$

$$V = 5$$



Rabin Karp

V U A T S

$$1 + 4 = 5$$

Spurious Hit

T S

$$2 + 3 = 5$$

2

Values

$$U = 1$$

$$T = 2$$

$$S = 3$$

$$A = 4$$

$$V = 5$$



Rabin Karp

V U A T S
4 + 2 = 6

T S
2 + 3 = 5

³ Values

U = 1

T = 2

S = 3

A = 4

V = 5



Rabin Karp

V U A T S

4

Values

$$2 + 3 = 5$$

Matched !

T S

$$2 + 3 = 5$$

$$U = 1$$

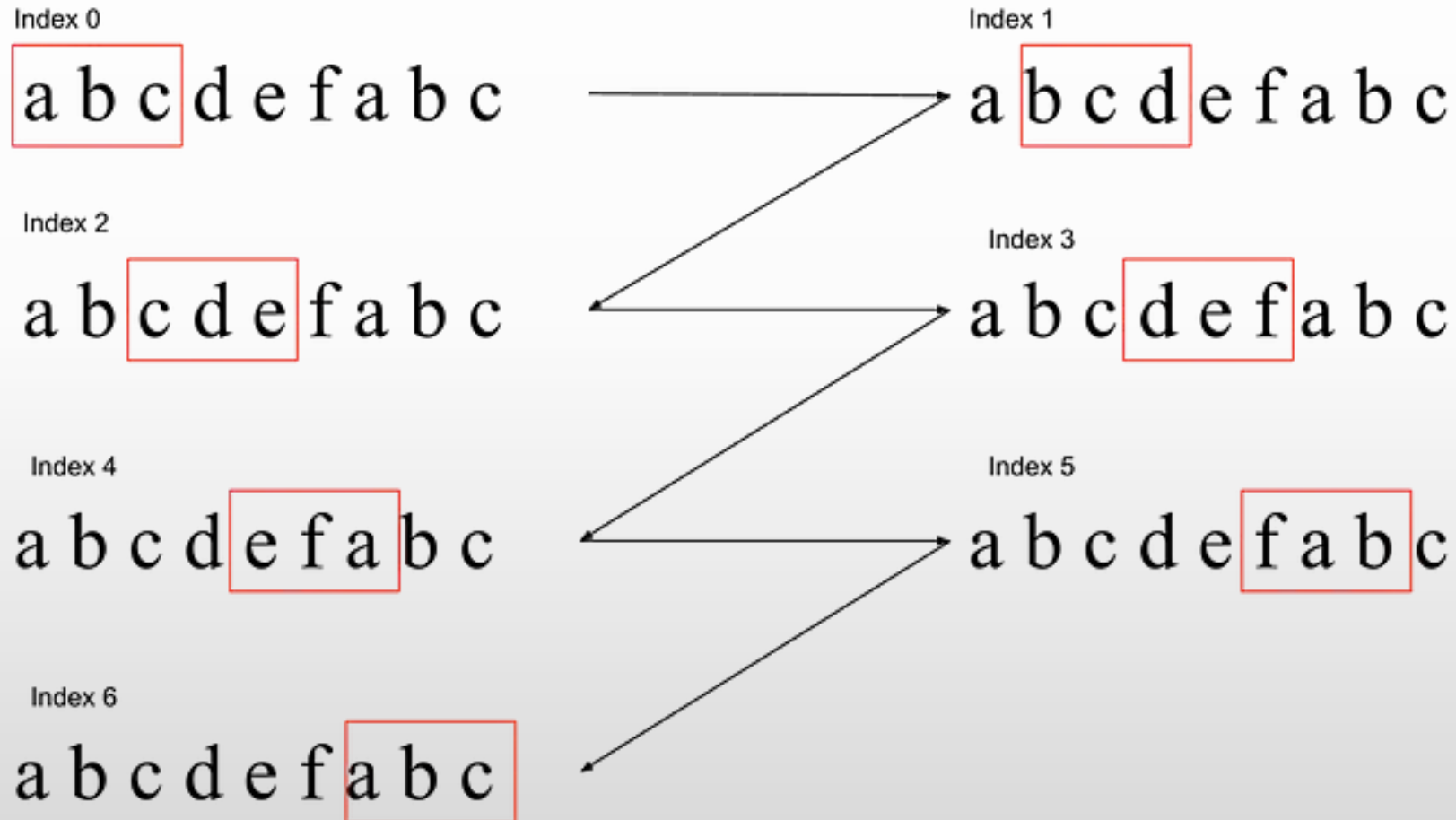
$$T = 2$$

$$S = 3$$

$$A = 4$$

$$V = 5$$

Rabin Karp





Karşılaştırma

Algoritma	Temel fikir	Ön işleme	Ortalama durum	En kötü durum
Naive	Her konumu dener	Yok	Veri yapısına bağlı	$O(nm)$
KMP	LPS tablosu	Var	$O(n + m)$	$O(n + m)$
Boyer-Moore	Sağdan sola karşılaştırma	Var	Çok hızlı/pratik	$O(nm)$
Rabin-Karp	Hash karşılaştırması	Var	$O(n + m)$	$O(nm)$

String Compression





Sıralı Sıkıştırma Kodlaması (RLE)

- RLE, metin verilerini sıkıştırmak için kullanılır.
- Ardışık tekrar eden karakterleri bir kodla yer değiştirir.
- Metin sıkıştırmada kullanılan basit ve etkili bir yöntemdir.
- Tekrar eden karakterlerin sayısı az ise etkisiz olabilir.



RLE Sıkıştırma

- Sıkıştırma Süreci:
 - Metin: "AAAAABBBCCCCDDD"
 - Sıkıştırılmış Hali: "5A3B4C3D"
 - Ardışık tekrar eden karakterlerin sayısı ve kendisi ile temsil edilmesi.



Sıkıştırma Adımları

- Metin soldan sağa taranır.
- Ardışık tekrar eden karakterler sayılır.
- Tekrar eden karakterler için sayı ve karakter bir araya getirilir.
- Bu bilgiler sıkıştırılmış metin olarak saklanır.



Algoritma Karmaşıklığı

- RLE algoritması, girdi metni tek bir kez tarar.
- Ardışık tekrar eden karakterlerin sayısını kaydeder.
- Bu nedenle, algoritmanın zaman karmaşıklığı, $O(n)$.



RLE Sıkıştırma

W1B1W3B1W1

B3W1B3

B7

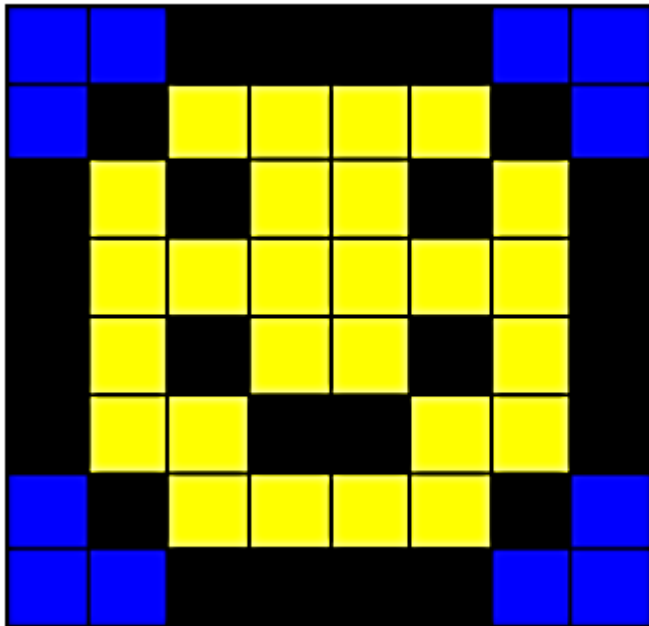
W1B5W1

W2B3W2

W3B1W3



Pixel level RLE



RGB colour values			
	R	G	B
	0	0	255
	0	0	0
	255	255	0



Pixel level RLE

Count	R	G	B	Count	R	G	B	Count	R	G	B
2	0	0	255	4	0	0	0	3	0	0	255
1	0	0	0	4	255	255	0	1	0	0	0
1	0	0	255	1	0	0	0	1	255	255	0
1	0	0	0	2	255	255	0	1	0	0	0
1	255	255	0	2	0	0	0	6	255	255	0
2	0	0	0	1	255	255	0	1	0	0	0
2	255	255	0	1	0	0	0	1	255	255	0
2	0	0	0	2	255	255	0	2	0	0	0
2	255	255	0	1	0	0	0	1	0	0	255
1	0	0	0	4	255	255	0	1	0	0	0
3	0	0	255	4	0	0	0	2	0	0	255





Lempel-Ziv-Welch (LZW) Sıkıştırma Algoritması

- LZW, metin sıkıştırma için kullanılan etkili bir algoritmadır.
- Tekrar eden dizgileri tanımlayarak sıkıştırma yapar.
- Metin sıkıştırmada yaygın olarak kullanılır.
- Sözlük boyutu büyüdükçe performansı düşer.



LZW Sıkıştırma

- Sıkıştırma Süreci:
 - Metin: "ABABCABABCA"
 - Sıkıştırılmış Hali: "0 1 2 4 3 5"
 - Metin içindeki tekrar eden dizgilerin bir indeksle temsil edilmesi.



Sıkıştırma Adımları

- Metin soldan sağa taranır.
- İndekslerle tanımlanan dizgilerin bulunduğu bir sözlük oluşturulur.
- Sözlükte olmayan yeni dizgiler eklenir.
- Dizgiler sıkıştırılmış metin olarak depolanır.



Lempel-Ziv-Welch

A T G A T C A T G A G

code	word	out
------	------	-----

0	A	
---	---	--

1	T	
---	---	--

2	G	
---	---	--

3	C	
---	---	--

4	AT	0
---	----	---

5	TG	1
---	----	---

6	GA	2
---	----	---

7	ATC	4
---	-----	---

8	CA	3
---	----	---

9	ATG	4
---	-----	---

10	GAG	6
----	-----	---

		2
--	--	---



Lempel-Ziv-Welch

	Output	Dict.													
<table><tr><td>a</td><td>a</td><td>b</td><td>a</td><td>a</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>b</td></tr></table>	a	a	b	a	a	c	a	b	c	a	b	c	b	(0, a)	1 = a
a	a	b	a	a	c	a	b	c	a	b	c	b			
<table><tr><td>a</td><td>a</td><td>b</td><td>a</td><td>a</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>b</td></tr></table>	a	a	b	a	a	c	a	b	c	a	b	c	b	(1, b)	2 = ab
a	a	b	a	a	c	a	b	c	a	b	c	b			
<table><tr><td>a</td><td>a</td><td>b</td><td>a</td><td>a</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>b</td></tr></table>	a	a	b	a	a	c	a	b	c	a	b	c	b	(1, a)	3 = aa
a	a	b	a	a	c	a	b	c	a	b	c	b			
<table><tr><td>a</td><td>a</td><td>b</td><td>a</td><td>a</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>b</td></tr></table>	a	a	b	a	a	c	a	b	c	a	b	c	b	(0, c)	4 = c
a	a	b	a	a	c	a	b	c	a	b	c	b			
<table><tr><td>a</td><td>a</td><td>b</td><td>a</td><td>a</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>b</td></tr></table>	a	a	b	a	a	c	a	b	c	a	b	c	b	(2, c)	5 = abc
a	a	b	a	a	c	a	b	c	a	b	c	b			
<table><tr><td>a</td><td>a</td><td>b</td><td>a</td><td>a</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>b</td></tr></table>	a	a	b	a	a	c	a	b	c	a	b	c	b	(5, b)	6 = abcb
a	a	b	a	a	c	a	b	c	a	b	c	b			



Lempel-Ziv-Welch

	Output	Dict.
a a b a a c a b a b a c b	112	256=aa
a a b a a c a b a b a c b	112	257=ab
a a b a a c a b a b a c b	113	258=ba
a a b a a c a b a b a c b	256	259=aac
a a b a a c a b a b a c b	114	260=ca
a a b a a c a b a b a c b	257	261=aba
a a b a a c a b a b a c b	261	262=abac
a a b a a c a b a b a c b	114	263=cb



SON